

STAT 6910 Advanced R

Project 1

by

Jürgen Symanzik

Date: November 25, 2013

Due Date: Monday, December 9, 2013, 11:59pm (by e-mail)

UTAH STATE UNIVERSITY

Logan, UT

Fall 2013

Contents

1	General Instructions	1
2	Project Description	2
	Appendices	4
	Appendix A R Code for Project 1	4

1 General Instructions

You have to work in small groups of exactly two students on this project. Unless you have a strong request to work with someone else for this project, you will work with the same partner as in the previous homeworks. Any requests to work with a new group partner must be made by 10am (MT) on Tuesday, November 26, 2013.

I would suggest that you first work independently on this project and then compare your results with your partner. You can submit only one final version!

You have to submit **three** files via e-mail: The `.Rnw` file that contains the $\text{\LaTeX}$ and R/Sweave (or R/knitr) code, the `.R` file that contains the complete R functions, and the resulting `.pdf` file prepared on your side. Your files should be named `LastName1_LastName2_Project01.Rnw`, `LastName1_LastName2_Project01_RCode.R`, and `LastName1_LastName2_Project01.pdf`.

Your R code has to be fully reproducible! You cannot include any constants in your $\text{\LaTeX}$ text. Rather, you have to work with `\Sexpr` to extract numeric values from your R code. I will likely make changes to your R code, e.g., change some of the numerical values, to check whether your results properly get adapted to these new values.

Use this `.Rnw` file as the basis for your answers. Update the title page, e.g., fill in your names and indicate when this project gets submitted. However, all of your R functions should be included in a separate `.R` file. Include this file in the appendix of your report. In the main file, include this `.R` file via the `source` command. Only show the function calls that are needed to produce the numerical results and figures in the main text.

Here is an example: The R code is stored in the external file called `Project_01_RCode.R` and is listed in Appendix A. In the main `.Rnw` file, we have to do the following to be able to use this R code:

```
> source("Project_01_RCode.R")
> myFactorial1(10)
```

```
[1] 3628800
```

Or, include the result into the $\text{\LaTeX}$ text via $5! = 120$.

Before you submit your project, check once more that you follow all recommendations from Google's R Style Guide (see <http://google-styleguide.googlecode.com/svn/trunk/Rguide.xml>).

For general questions related to this project, please use the corresponding discussion board in Canvas! I will try to reply as quickly as possible. Moreover, if one of you knows an answer, please post it. It is fine to refer to web pages and R commands, but do not provide the exact R command! This will be the task for each individual group.

2 Project Description

In this project, you will have to deal with e-mails that were previously classified as spam by spam detection software called SpamAssassin. We want to reassess the decisions made by SpamAssassin to filter out all of these e-mails as being spam e-mails. There will be one training file for your R code called `Junkmail`.

You have to solve the following tasks. In your answer, provide a brief textual explanation for each task. Include the function call (but not the function itself) and the result(s).

For some tasks, you have to decide what to do exactly. Explain your choice(s).

1. Read in the e-mail data from the external `Junkmail` file and create a data frame with the following columns: `eMailID` (from 1 through n), `fromEMail` (note that some e-mails have more than one from field — one is fake), `fromTrueName`, `date`, `dayOfWeek`, `time`, `subject`. Some of these may be empty or missing.
2. For each e-mail, create a list that consists of two components: The total spam points assigned by SpamAssassin and a data frame with two columns where the first column contains the type of the spam indicator (such as “From: does not include a real name”) and the second column contain the score for this indicator (such as 0.8). The elements should be accessible via the same `eMailID` as used in part 1.
3. For each e-mail, create a 2nd list that consists of three components: The first one is a vector that contains each row of the e-mail header as a separate string. The second one is a vector that contains each row of the e-mail body as a separate string. Note that this may be html code. The third one is a single number that indicates the number of attachments to this e-mail. The elements should be accessible via the same `eMailID` as used in part 1.

Use your data structures and write any necessary functions to answer the following questions. The numbers below are examples that should be used in your report, but your functions should also be tested internally with different values. Ideally, the person who did not write the function should do the testing! There should be some basic error checking in your functions.

The function calls and results for the values below should be placed in the main text of your report.

4. How many e-mails are there overall in this file?
5. How many e-mails do not contain a real name?
6. What are the 3 most frequent senders, based on the `fromEMail` addresses? If there is a tie, list all other senders as well that occur the same number of times as the 3rd one.
7. During which time period were these e-mails received? List the first and last date as “mm/dd/yyyy”. Do not use the format that is used in the e-mails.
8. Create a bar chart that shows the distribution of these e-mails during the seven days of the week. Sort from day with most e-mails to day with least e-mails received.
9. Create a histogram that shows the distribution of these e-mails over the 24 hours of a day. Work with 2-hour wide intervals, starting at midnight.

10. What are the 20 most frequent character-based keywords (such as buy, purchase, re, free, fwd) in the subject line across all e-mails? Translate all keywords to lower case. Remove all numbers, spaces, and any other non-alphabetic symbols. If there is a tie, list all other keywords as well that occur the same number of times as the 20th one.
11. In a graphic of your choice, display the distribution of the total spam points of all e-mails.
12. What are the 10 most frequent spam indicators? If there is a tie, list all other spam indicators as well that occur the same number of times as the 10th one.
13. Return the eMailID's of the e-mails with the 5 lowest total spam points. Which are those? No need to break any ties here. Just use the first one in your list in case of a tie.
14. Write a function called `InspectEmail` that has the following arguments:
 - `eMailID` a single e-mail ID
 - `part` this could be "header", "body", or "attachment"
 - `numLines` the number of lines to be shown (will be ignored for attachments)
 This function should show the first `numLines` of the `part` or simply indicate the number of attachments. If the actual number of lines in `part` is less than `numLines`, show all lines from `part`. Test your function for 6 different scenarios (two for each `part`).
15. For the 5 e-mails with the lowest total spam points, look at 20 lines from the "body". Do all of these bodys look like spam to you? Be specific!
16. So, what do you think of the overall performance of the SpamAssassin software? Base your conclusions on your previous results.

A major part of software development is to (manually) verify the results for several test cases!!! You can open Junkmail in any text editor of your choice and verify whether your R code produces what it is supposed to do! Note that what appears to be a space in a text editor may not actually be a space in the file. Also be aware that there are often hidden spaces at the end of a line!

Appendix A R Code for Project 1

Here is the R code for Project 1:

```
myFactorial1 = function(n) {  
  return(factorial(n))  
}  
  
myFactorial2 = function(n) {  
  return(prod(1:n))  
}  
  
myFactorial3 = function(n) {  
  if (n == 0) {  
    return(1)  
  } else {  
    result = 1  
    for (i in 1:n) {  
      result = result * i  
    }  
    return(result)  
  }  
}  
  
myFactorial4 = function(n) {  
  if (n == 0) {  
    return(1)  
  } else {  
    result = 1  
    i = 1  
    while (i <= n) {  
      result = result * i  
      i = i + 1  
    }  
    return(result)  
  }  
}  
  
myFactorial5 = function(n) {  
  if (n == 0) {  
    return(1)  
  } else {  
    return(n * myFactorial5(n - 1))  
  }  
}  
  
myFactorial6 = function(n) {  
  if (n == 0) {  
    return(1)  
  } else {  
    return(myFactorial6(n - 1) * n)  
  }  
}
```